

Cron-as-a-Service: Complete Project Plan

Tech stack, architecture, plans and pricing, payments, security, costs, and a build timeline, based on every decision finalized so far.

Status: working name and domain are still to be decided. Platform limits, fees, and competitor terms were checked recently but change often; items marked “verify” need a check against the official source before you rely on them.

1. Product Summary

A hosted service that calls a user’s URL on a schedule, tells them when it fails, and keeps a history. The free tier works as uptime monitoring plus keep-alive pings every 10 minutes. Paid tiers add per-minute runs and the features that make a scheduler dependable: retries, richer alerts, success conditions, heartbeat monitoring, and longer history.

Who it is for

- Students and indie developers on free hosts that put idle apps to sleep, who want the app woken and want to know when it is down.
- Developers who need scheduled HTTP calls (webhooks, syncs, cleanups) but whose host has no scheduler or limits it, for example free hosts where cron is a paid feature.
- Small teams who want one dashboard for scheduled work instead of cron spread across services.

Why someone would pay

A plain “ping my URL” feature is a commodity, and free tools already do it. Developers who know the self-ping trick (a timer inside the app calling its own health route) will use it. That trick fails in exactly the situations that hurt: after a restart or crash nothing is left running to wake the app, it does not work on serverless hosts, and a dead app cannot alert you about itself. The paid value is everything an outside observer can do:

- Alerts that reach you when a job fails, with the response details.
- Retries and success conditions, so a 200 with the wrong body still counts as a failure.
- Heartbeat monitoring, where the user’s own job checks in and you alert when it stops.
- Per-minute scheduling, POST/PUT/DELETE with headers and body, and searchable history.
- INR and UPI pricing and GST invoices, which dollar-priced tools do not offer.

Honest competitive picture

Services such as cron-job.org and UptimeRobot already offer free pings and monitoring, some at faster intervals. The free tier is therefore acquisition, not a differentiator. Validate demand before building much (section 12).

2. Decisions Finalized

Area	Decision
Infrastructure	Single platform: Cloudflare Cron Trigger (every minute) + one Worker + D1. Dashboard on Cloudflare Pages. No second database for now.
Free plan	Fixed 10-minute GET ping, 3 jobs per user, global cap of about 300 free jobs with a waitlist beyond it, email alerts on down and recovered, last 50 results.
Paid plans	Pro and Team: per-minute runs, all HTTP methods, retries, more alert channels, success conditions, heartbeat monitoring, longer history.
Pricing	Pro ₹249/month or ₹2,499/year; Team ₹599/month. Friends coupon (about 20% off) brings Pro to ₹199/month or ₹1,999/year. Starting points, to be tested with strangers.
Scheduling method	Slot-based: each job has an interval and a slot, and is due when $(\text{minute} \bmod \text{interval}) = \text{slot}$. Successful pings write nothing to the database.
Queues	Not used on the free plan (daily allowance too small). Add after upgrading.
Payments	Razorpay Subscriptions (UPI AutoPay and cards), INR only at launch, webhook-driven plan state.
Email	Brevo free tier on your own domain for launch (Resend as fallback); Gmail API only for early testing. Sending sits behind one function.
Upgrade trigger	Move to Workers Paid (about \$5/month) when the first Pro customer pays.
Order of work	Validate demand first, build the free tier, then alerts, then Razorpay and Pro.

3. Plans and Pricing

Prices below are recommended starting points, not final. Test them in your validation conversations and adjust before launch.

	Free	Pro	Team
Price	₹0	₹249/month or ₹2,499/year	₹599/month
Interval	Fixed 10 minutes	Down to 1 minute	Down to 1 minute
Active jobs	3	25	100
HTTP methods	GET only	GET, POST, PUT, DELETE, headers, body	Same as Pro
Alerts	Email on down and recovered	Email, Telegram, Discord, Slack, webhook	Same, plus team routing
Retries	None	Configurable, with backoff	Configurable
History	Last 50 results	30 days	90 days
Success conditions	No	Status, body text, JSON field	Same as Pro
Heartbeat monitoring	No	Yes	Yes
Members, API keys	No	No	Yes
Support	Docs	Email	Priority email

Pricing logic

- **Why ₹249 for Pro:** low enough for a student or indie developer to pay without approval, and high enough that two subscribers roughly cover the Cloudflare upgrade (three at the ₹199 friends price). The yearly price is ten months of the monthly price.
- **Why Team is ₹599:** it adds members, API keys, 100 jobs, and 90 days of history for about 2.4 times the Pro price, which keeps it within reach of a small team or a student group. It is easy to change before launch.
- **Headroom for GST:** if you register and show GST-inclusive prices, 18% comes out of each payment, so ₹249 leaves about ₹211. A lower list price would leave much less.
- **Why Free is GET-only:** it limits abuse and keeps the free tier cheap.
- **Founding-member offer:** the friends coupon below doubles as this: a locked-in lower price for the first 20 to 30 users in exchange for feedback.
- **Trial:** decide later whether to offer a short Pro trial. Razorpay subscriptions can be set to start in the future, which supports this (verify).
- **GST:** decide with a chartered accountant whether prices are shown inclusive or exclusive of GST, and when registration is required. Registration is required above a turnover threshold (I believe ₹20 lakh for services) unless other rules apply; confirm for your situation.

Friends coupons

Friends and early supporters get a coupon worth about 20% off, which is ₹50 on the monthly plan. That brings Pro to ₹199/month or ₹1,999/year.

- **Plans in Razorpay:** pro_monthly (₹249), pro_yearly (₹2,499), pro_monthly_friends (₹199), pro_yearly_friends (₹1,999), team_monthly (₹599). Your coupon table decides which plan id checkout uses.
- **Why separate plans:** Razorpay's own offers feature may discount only some billing cycles on recurring payments. Verify before relying on it. Separate plans make the discount permanent and predictable.
- **Code rules:** one code per friend, or a shared code with a redemption cap; an expiry date; valid on Pro only; one redemption per account.
- **Permanent or temporary:** recommended permanent while the subscription stays active, limited to the first 20 to 30 codes. If someone cancels and later returns, they pay the list price unless you decide otherwise.
- **When to count a redemption:** when the subscription activates (the webhook), not when checkout opens, so abandoned checkouts do not use up codes.
- **Do not treat friends as validation.** Friends may pay partly out of friendship. Test the ₹249 price on strangers (section 12). If strangers push back, make ₹199 the public price and keep coupons for launch promotions.

4. Tech Stack

Layer	Choice	Why
Language	TypeScript everywhere	One language for Worker, API, and dashboard.

Layer	Choice	Why
Scheduler	Cloudflare Cron Trigger, every minute	Free, no server to manage; one minute is its finest granularity.
Execution engine	One Cloudflare Worker (scheduled handler)	Wakes each minute, reads due jobs, pings, writes state changes.
Database	Cloudflare D1 (SQLite)	Bound to the Worker; generous on the paid plan.
API	Hono on Workers	Small, fast, built for Workers.
Dashboard	React + Vite on Cloudflare Pages	Free hosting, deploys from GitHub.
Auth	GitHub and Google sign-in; sessions stored in D1	Your audience has GitHub accounts; avoids sending login emails.
Validation	zod	Validate every input, especially URLs.
Cron expressions (phase 2)	croner or cron-parser	Do not hand-roll cron or timezone logic.
Payments	Razorpay Subscriptions via REST (fetch)	The Node SDK may not run well on Workers.
Email	Transactional provider on own domain	Deliverability and branding (section 8).
Bot protection	Cloudflare Turnstile	Free signup protection.
Deploy and CI	Wrangler + GitHub Actions	Repeatable deploys, test on every push.
Observability	Workers Logs, per-tick stats table, public status page	Know when your own service is failing before users tell you.

5. Architecture and Scheduling

Flow

1. Cloudflare fires the Worker's scheduled handler once per minute.
2. The Worker computes the current minute number and queries D1 for active jobs whose slot matches.
3. It pings up to about 40 URLs, 6 at a time, each with a timeout.
4. It works out which jobs changed state (up to down, or down to up) and writes only those, plus one summary row for the tick, in a single batch.
5. It sends alerts for state changes only.

Tick handler sketch

```

async scheduled(ctl, env) {
  const minute = Math.floor(ctl.scheduledTime / 60000);
  const { results: jobs } = await env.DB.prepare(
    "SELECT id, url, mode, timeout_ms, fail_streak, state FROM jobs
     WHERE status = 'active' AND (?1 % interval_min) = slot
     ORDER BY plan_priority DESC LIMIT 40"
  ).bind(minute).all();

```

```

const outcomes = await pingInBatches(jobs, 6); // AbortSignal.timeout per request
const changes = outcomes.filter(isStateChange); // 3 fails => down, 1 success => up

await env.DB.batch([
  ...changes.map(updateJobState),
  insertTickStats(minute, outcomes),           // one summary row per tick
]);
await sendAlerts(env, changes);                // state changes only
}

```

Why this design

- **No writes on success.** The usual “update next_run after every run” design would exhaust the free 100,000 daily row writes. Here, successful pings change nothing.
- **Two database calls per tick**, well within the free per-invocation limits, leaving subrequests for pings.
- **Stagger.** When a job is created, assign it the least-loaded slot (0 to 9 for a 10-minute job) so work spreads evenly across minutes.

Free-tier capacity (Workers Free)

Limit	Effect on the design
50 subrequests per invocation	About 40 pings per tick. This is the main cap.
6 simultaneous outbound connections	Pings go in batches of 6; slow wake-ups make ticks long but allowed.
100,000 requests per day	Ticks use 1,440. Plenty of room.
10 ms CPU per invocation	Keep the handler lean; waiting on responses does not count.
D1: 5M reads, 100K writes per day	Reads are small; the write-free design avoids the write cap.
Queues: 10,000 operations per day	About 3 operations per message, so too small. Do not use on free.

Result: if about 10 of the 40 pings per tick are reserved for early Pro per-minute jobs, the remaining 30 across 10 slots support roughly 300 free jobs, or about 100 users at 3 jobs each. After upgrading to Workers Paid the subrequest cap rises to the thousands, and Queues becomes affordable for fan-out.

Handling sleeping hosts

- Use a 45 to 60 second timeout for keep-alive jobs, since a sleeping host can take that long to wake.
- **Keep-alive mode:** any HTTP response, even a 404, proves the host is awake. **Monitor mode:** require a 2xx or the configured success condition.
- Mark a job down after 3 consecutive failures; send one down email and one recovered email.
- Auto-pause a job that has failed continuously for 7 days and tell the user.

Known trade-offs

- If a tick is skipped, that slot waits another 10 minutes, a 20-minute gap that could let a 15-minute-sleep host fall asleep. Pinging every 5 minutes covers this but halves capacity. Start at 10 and watch how late ticks run.

- If more than 40 jobs are due in one minute, the excess waits. Record an overflow metric so you know when to rebalance or upgrade.

6. Data Model

Table	Key columns	Purpose
users	id, email, auth_provider, provider_id, plan, created_at	Accounts and current plan.
sessions	id_hash, user_id, expires_at	Login sessions.
jobs	id, user_id, name, url, method, headers_enc, body, mode, interval_min, slot, timeout_ms, status, state, fail_streak, last_state_change, plan_priority	The job definition and its current state.
job_events	id, job_id, at, kind (down, recovered, paused), http_status, error, duration_ms	State changes and failures only on Free; full run history on Pro.
tick_stats	minute, jobs_run, ok, failed, overflow, duration_ms	One row per tick for health and capacity tracking.
alert_channels	id, user_id, type, target_enc, verified	Where alerts go.
heartbeats	job_id, token, expected_every_min, grace_min, last_seen	Heartbeat monitoring (Pro).
subscriptions	user_id, razorpay_sub_id, plan, status, current_end, cancel_at_period_end	Billing state, written by webhooks only.
payment_events	event_id (unique), type, received_at	Idempotency for webhooks.
coupons	code, plan_scope, discount_plan_id, max_redemptions, redeemed, expires_at, active	Friends and promotional codes.
coupon_redemptions	code, user_id (unique per code), subscription_id, redeemed_at	Who used which code, written on activation.
api_keys	id, user_id, hash, last_used	Team plan API access.

Index jobs on status and interval/slot. Remember that D1 counts rows scanned, not rows returned, so an unindexed query can burn through the daily read allowance. Prune old events on a schedule.

Scope note: launch Pro with interval-based schedules (every 1, 5, 10, 15, 30, 60 minutes, daily). Full cron expressions with timezones and DST handling come in phase 2, using a next_run column and a cron library.

7. Security and Abuse Prevention

You are letting strangers make your infrastructure call arbitrary URLs. Treat abuse prevention as a launch requirement.

- **URL rules:** HTTPS only on Free; block localhost, private and reserved IP ranges, IP-literal hosts, and non-HTTP schemes; do not follow redirects blindly.
- **Limits:** 3 jobs per Free user, a minimum interval per plan, a cap on jobs hitting the same host, and per-user API rate limits.
- **Signup protection:** verified email or OAuth, Turnstile, and rate limits on signup.

- **Secrets:** encrypt header values and alert targets at rest (AES-GCM with a key stored as a Worker secret); never log or display them in plaintext.
- **Request signing:** sign outgoing Pro requests with an HMAC header so users can verify the source, and send an identifying User-Agent.
- **Responses:** do not store full response bodies; truncate and cap stored size, and cancel the body on keep-alive pings.
- **Circuit breaker:** track daily D1 usage; near about 80% of the daily limit, drop non-essential logging first and keep pings running. Free-plan queries stop when a daily limit is hit, which would take the whole service down until midnight UTC.
- **Abuse process:** an acceptable-use policy, an abuse contact address, and the ability to suspend an account immediately.
- **Authorization:** every query filtered by user_id; test for cross-account access.

8. Alerts and Email

- **Alert on state change only:** one down message after 3 failures, one recovered message. Never one email per failed ping.
- **Channels:** email at launch. Telegram bots, Discord and Slack webhooks, and generic webhooks are simple fetch calls and can follow in Pro.
- **Gmail:** SMTP from a Worker is unreliable, since common SMTP libraries depend on Node networking and outbound SMTP ports are restricted. Use the Gmail API over HTTPS for early testing only. A free Gmail account has a daily sending cap and can be suspended for bulk-looking mail.
- **Provider: Brevo free tier.** Use it on your own domain with SPF, DKIM, and DMARC set up, and call its HTTP API from the Worker with fetch. It advertises about 300 emails per day, which is enough while you have tens of users because alerts fire only on state changes. Resend (about 3,000 per month, capped at 100 per day) is the fallback.
- **Watch for:** a host-wide outage can trigger many alerts at once (each affected user gets a down email and later a recovered email), so a daily cap can be reached; new accounts may need validation before sending; the free plan may add sender branding; and sources disagree on current free-plan terms, so confirm on Brevo's pricing page.
- **Reduce volume:** send one email per user per tick that lists all their affected jobs, and carry any overflow to the next day.
- **Keep sending behind one function** so changing provider is a one-file change.
- **Inbound:** Cloudflare Email Routing can forward your support address to your inbox for free.

9. Payments with Razorpay

Account setup

1. Create a Razorpay account and use test mode first; it costs nothing.
2. Prepare for live activation early: KYC, bank account, and a live website with terms of service, privacy policy, refund policy, and contact details. Approval can take time.
3. Create plans in the dashboard: Pro monthly and yearly, Pro friends monthly and yearly, and Team monthly (section 3 lists the prices).

Checkout flow

1. The user clicks upgrade and may enter a coupon. Your API validates it, picks the standard or friends plan id, creates a Razorpay subscription with the REST API, and returns its id.
2. The dashboard opens Razorpay Checkout with that subscription id. The user pays by UPI AutoPay or card.
3. The client-side success callback is only a hint. Do not grant a plan from it.
4. Razorpay sends webhooks to a Worker route. The webhook is the source of truth for plan state.

Webhook handling

- Verify the signature header: HMAC-SHA256 of the raw request body with your webhook secret, using the Web Crypto API, compared in constant time.
- Store each event id in `payment_events` (unique) and ignore repeats, since webhooks can be delivered more than once.
- Return a 2xx quickly; do heavy work after recording the event.

Event (verify exact names in docs)	Action
<code>subscription.authenticated</code>	Record the mandate; wait for activation or the first charge.
<code>subscription.activated / charged</code>	Set plan active; extend <code>current_end</code> .
<code>payment.failed / subscription.pending</code>	Mark past due; send a payment-failed email; start a grace period.
<code>subscription.halted</code>	Downgrade to Free after the grace period.
<code>subscription.cancelled / completed</code>	Keep access until <code>current_end</code> , then downgrade.

Plan state and downgrades

- States: free, active, `past_due` (grace period of about 3 to 7 days), `cancelled-at-period-end`.
- On downgrade, keep the oldest 3 jobs active, pause the rest, and retain data for 30 days so an upgrade restores everything.
- Plan changes: the simplest approach is to cancel at period end and start the new plan, rather than prorating mid-cycle.
- Refund policy: a clear 7-day refund on the first payment builds trust for a new product.

Fees

Razorpay lists a 2% platform fee plus 18% GST on that fee, about 2.36% all-in, with no setup fee or annual maintenance charge. Recurring billing may add a further subscription fee of around 1% on top (sources differ; check Razorpay's pricing page). Plan for roughly 3 to 3.5% total.

Other notes

- Launch in INR only. International payments need separate enablement and can wait.
- Invoices: generate a GST-style invoice per payment (Razorpay invoices or your own) once you are registered.
- Store only Razorpay ids and plan state; never store card or bank details.

10. Pro Features in Detail

- **Retries:** on failure, retry once or twice with a short delay inside the same tick, then rely on the failure streak. Each retry costs a subrequest.
- **Success conditions:** allowed status codes, body contains text, or a JSON field equals a value.
- **Heartbeat monitoring:** each job gets a unique check-in URL. The user's own cron job calls it; the tick finds heartbeats overdue past interval plus grace and alerts. To limit writes, update `last_seen` at most once per minute per heartbeat.
- **POST, PUT, DELETE with headers and body:** encrypted at rest, with a size limit.
- **Run now and test run:** a button that runs the job once before saving; rate-limited.
- **Pause and resume, plus history search:** filter by status and date.
- **Team:** members and roles, API keys, audit log, longer retention.

Skip at first: a visual cron builder beyond something simple, a workflow editor, and SDKs in many languages.

11. Dashboard and Marketing

Pages

Landing and pricing, sign in, job list with status, create job (with a one-click “keep my app awake” template), job detail with uptime and recent results, alert settings, billing, docs, and a public status page for your own service.

Getting the first users

- Write short guides targeting the problems people search for, such as keeping a free-hosted app awake and scheduling jobs on hosts without cron.
- Share progress publicly (LinkedIn, GitHub, developer communities and subreddits, college developer groups) and ask for feedback rather than just announcing.
- Offer the founding-member discount to early adopters in exchange for conversations.

12. Validation Plan (Do This First)

1. Put up a one-page site describing the service with a waitlist form.
2. Talk to 15 to 20 developers who deploy on free hosts. Ask what they do today about sleeping apps and downtime, and what has failed. Do not pitch; listen.
3. Ask directly whether they would pay for reliable alerts and per-minute runs. Test a few price points (for example ₹149, ₹199, and ₹249 a month) with people who are not friends, and ask what would make them pay.
4. Decide: continue if at least a handful say yes with specific reasons; otherwise change the pitch or stop.

If validation fails, you have lost a couple of weeks and no money. That is the main reason to do it before building.

13. Costs and Unit Economics

Item	Cost	When
Cloudflare Workers Paid	About \$5/month minimum	On the first Pro customer
D1, Pages, Turnstile, Email Routing	₹0 within allowances	Always
Domain	Yearly fee (check price)	Before beta
Email provider	₹0 on a free tier at launch	Before launch
Razorpay	About 3 to 3.5% of each payment	Per payment
Accountant (GST, filings)	Varies	Before first sale

Illustrative monthly numbers, assuming about 3.5% payment fees, about ₹450 a month for the Cloudflare plan, and no GST registration. These are approximations, before income tax. The table shows all subscribers paying the list price, and all paying the friends price.

Pro subscribers	Revenue at ₹249	After fees and Cloudflare	Revenue at ₹199	After fees and Cloudflare
2	₹498	about ₹30	₹398	about -₹65
3	₹747	about ₹270	₹597	about ₹125
10	₹2,490	about ₹1,950	₹1,990	about ₹1,470
50	₹12,450	about ₹11,560	₹9,950	about ₹9,150
100	₹24,900	about ₹23,580	₹19,900	about ₹18,750

In practice you will have a mix of both prices, so your real figure sits between the two columns. Team subscribers at ₹599 add more per head. At the friends price you need three subscribers to cover Cloudflare; at the list price, two.

The first scaling pressure will be database writes from full Pro run history. Sample successful runs, keep failures in full, and prune on a schedule. Revisit the infrastructure at around 100 Pro subscribers.

14. Legal and Business Basics

- Publish terms of service, privacy policy, refund policy, acceptable-use policy, and contact details. Razorpay activation usually needs them.
- Collect the minimum personal data (email and provider id). India's data protection law applies to personal data you process; keep retention short and offer account deletion.
- You can usually begin as an individual or sole proprietor and move to a company later. Confirm the right structure, GST position, and tax treatment with a chartered accountant before your first sale.
- Say in your terms that users are responsible for having permission to ping the URLs they add, and for checking their host's rules about keep-alive pings.

15. Build Timeline

Assumes about 8 to 10 focused hours a week, which leaves room for classes and other commitments. Adjust the dates, not the order.

Phase	Weeks	Deliverables
0. Validate	1	Waitlist page, 15 to 20 conversations, go or no-go.
1. Free-tier core	2 to 4	Worker tick, D1 schema, slot scheduling, ping logic and state machine, OAuth sign-in, minimal dashboard.
2. Alerts and guardrails	5 to 6	Email alerts, Turnstile, URL rules, quotas, circuit breaker, legal pages, status page.
3. Private beta	7	20 users, collect feedback, watch capacity and tick lateness.
4. Payments and Pro v1	8 to 10	Razorpay test mode and KYC, webhooks, plan state, POST/headers/body, retries, more channels, longer history.
5. Public launch	11 to 12	Pricing page, founding-member offer, first guides and posts, upgrade Workers on the first payment.
6. Phase 2	After	Success conditions, heartbeat monitoring, cron expressions and timezones, Team plan, API keys, Queues.

Illustrative targets

For the first 60 days after launch: around 100 sign-ups, 10 or more active free users, and 3 or more paying. Treat these as a gut check, not a promise. If beta users do not return or no one will pay, stop or change the pitch before building Pro.

16. Risks

Risk	Impact	Mitigation
Weak demand; free competitors	No paying users	Validate first; compete on alerts, reliability, India pricing; keep scope small.
Free-tier limits reached	Service stops until midnight UTC	Write-free design, global cap, circuit breaker, quotas, usage dashboard.
Abuse of outbound requests	Account suspension, harm to third parties	URL rules, Turnstile, per-host caps, GET-only Free, fast suspension.
Missed or late ticks	Hosts fall asleep; false alarms	Monitor tick lateness; consider 5-minute pings later.
Email deliverability	Alerts land in spam	Own domain with SPF, DKIM, DMARC; transactional provider.
Razorpay approval delay	Cannot charge	Start live activation early; keep test mode working meanwhile.
Platform or terms change	Limits or prices shift	Keep scheduling and email behind small interfaces; re-check limits quarterly.
Time and attention	Project stalls	Phase gates; keep Free launch small; stop if validation fails.

17. Open Decisions and Items to Verify

Still to decide

- Product name and domain.
- Final prices (Pro, Team, yearly, friends coupon) after testing with strangers, whether to offer a trial, and GST inclusive or exclusive pricing.
- Whether the friends discount is permanent or time-limited, and how many codes to issue.
- Business structure and GST timing, with a chartered accountant.
- Whether Pro launches with a small early-access cap until the Workers upgrade is paid for.

Verify before relying on them

- Cloudflare Workers, D1, and Queues free-tier limits, and how D1 queries count toward per-invocation limits.
- Render, Vercel, and other hosts: sleep behavior, free cron limits, and whether keep-alive pings are allowed.
- Razorpay fees for Subscriptions and UPI AutoPay, webhook event names, and signature details.
- Competitor free tiers (cron-job.org, UptimeRobot) so your positioning stays accurate.
- Brevo free-tier limits and terms: daily cap, account validation, and any sender branding.